UCS 503 Software Engineering Project Report

Proposal to Final

How to Write a Project Report

Submitted by:

102XXXXXXX Name Title
102XXXXXXX Name Title
102XXXXXXX Name Title
102XXXXXXX Name Title

Group: 3XXX BE Third Year, CSE

Submitted to:

Dr. NAME OF THE ADVISOR

Mid-Semester Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

Chapter 1

Proposal to Project Report

1.1 Overview

Transforming your proposal into a final project report is an incremental process of replacing **intentions** with **results**. Since a proposal serves as a roadmap, the final report becomes the record of the journey taken.

1.1.1 Expanding the Core Structure

To move from proposal to report, you must shift your perspective from what you *will* do to what you *have* done.

- **Introduction & Objectives:** Keep these sections but update them to reflect the final state of the project. If your objectives shifted during development, explain why.
- **Methodology to Implementation:** In a proposal, the methodology is a plan. In a report, this expands into a detailed **Implementation** section. For STEM projects, this includes your final system architecture, code modules, and integration workflows.
- **Results & Analysis:** This is an entirely new section. Replace your "Deliverables" list from the proposal with actual raw data, prototypes, and analyzed findings.
- **Conclusion & Future Work:** Summarize the impact of your results and suggest how others might improve upon your work.

1.1.2 Transitioning STEM Technical Content

For technical projects like AR or 3D modeling, your methodology evolves into technical documentation.

- **From Plan to Architecture:** Move from specifying intended tools (e.g., ARCore, Unity) to documenting the final **System Architecture** used.
- **Documenting Development:** Expand on how you handled surface detection, asset management, and UI design. Replace "proposed" scaling methods with documentation of your actual 1:1 metric scaling implementation.

- **Validation:** Your proposal lists testing goals; your report must provide the actual results of **Unit Testing**, **Performance Benchmarking** (like actual FPS achieved), and **User Acceptance Testing** (UAT).

1.1.3 Incremental Development Steps

Phase	Proposal	Evolution
Initial	SMART Objectives	Validated goals; noted deviations.
Mid-Project	Methodology & Tools	Detailed System Design and code structure.
Execution	Proposed Timeline	Actual log of milestones reached.
Final	Anticipated Risks	Evaluation & Testing results.

1.1.4 Pro-Tips for the Final Report

- **Density Over Fluff:** Just as in the proposal, professors value clear, concise information over high word counts.
- **Visual Documentation:** Replace the proposal's flowchart or table with actual screenshots of your software, photos of your setup, or graphs of your data.
- **Addressing Constraints:** Revisit the constraints you identified (like lighting or safety) and discuss how you successfully mitigated them in the final build.

1.2 More Nuanced Perspective

Transforming an academic proposal into a final project report is an incremental process of replacing planned intentions with documented results. Based on the provided guide, here is a holistic overview of how to modify and improvise upon your document through to completion:

1.2.1 Shift from Intent to Execution

The most significant change is moving from a "roadmap" of what you intend to do to a record of what was actually achieved.

- **Methodology to System Architecture:** In a STEM project, your methodology evolves from a plan into a detailed description of your final **System Architecture** and **Tech Stack**.
- **Module Development:** For the final report, expand on how you implemented specific features like **Surface Detection**, **Asset Management**, and **UI Design**.

- **Timeline to Project History:** Convert your weekly task breakdown into a summary of project milestones, noting any shifts in scope or timing.

1.2.2 Add Validated Results and Testing

A proposal only anticipates outcomes, whereas a report must validate them through rigorous testing.

- **Performance Benchmarking:** Include technical data such as **Frames Per Second (FPS)** and **latency** measurements to prove system stability.
- **User Acceptance Testing (UAT):** Document the results of "drift" analysis—checking if virtual objects remain stable in their environment over time.
- **Unit Testing:** Provide a log of individual feature tests, such as verifying if rotation or placement logic worked as intended.

1.2.3 Refine and Expand Core Sections

- **Introduction/Problem Statement:** Refine this section (ideally 250–400 words) to ensure it accurately reflects the final problem addressed by your completed work.
- **Objectives:** Revisit your **SMART objectives**. In your report, state clearly whether each objective was met, partially met, or modified.
- **Resources:** Update your resource list to reflect what was actually used, such as specific **AR libraries** (ARCore, ARKit) or hardware like **LiDAR-enabled devices**.

1.2.4 Address Real-World Constraints

While the proposal mentions anticipated risks, the final report should discuss how you managed them during development.

- **Environmental Constraints:** Discuss how the final system performed in low-light conditions or on reflective surfaces.
- **Technical Challenges:** Detail how you solved issues like **coordinate mapping** (1:1 scaling) or **physics engine** clipping.

1.2.5 Final Formatting Adjustments

- **Tone:** Maintain a professional tone using active verbs to describe your completed actions.
- **Density:** Ensure the final document remains dense with information rather than "fluff", focusing on proving the project's academic or scientific soundness.
- **Visuals:** Replace the proposal's placeholder tables with final data visualizations, flowcharts of your actual technical workflow, and screenshots of the finished product.

1.3 Example Details

These are the "meat" of your final report and transition your proposal's "Methodology" into a documented reality.

1.3.1 Section: Technical System Architecture

In the proposal, you listed tools. In the report, you explain how they interact. You should replace your methodology flowchart with a high-level system diagram.

Drafting Template:

- **The Tech Stack:** "The project was developed using **Unity 2022.3** as the primary engine, utilizing the **ARCore SDK** for spatial mapping. C# was used for logic scripting, while **Blender** served as the pipeline for 1:1 metric-scale 3D modeling."
- **The Workflow:** Describe the data flow. "The system initiates by polling the device camera for feature points. Once a horizontal plane is detected via **Raycasting**, the coordinate system is anchored to a 0,0,0 origin point to ensure stability."

1.3.2 Section: Module-by-Module Implementation

Break down the "doing" part of your project into logical modules. This proves the complexity of your work.

- **Module A: Environment Scanning & Plane Detection**
  - *Details:* How did you handle lighting? Did you use Depth APIs?

  - *Improvement:* "While the proposal suggested basic plane detection, the implementation utilized **Light Estimation APIs** to dynamically adjust the shaders on virtual objects, increasing realism."

- **Module B: Interaction Logic**
  - *Details:* How does the user move or rotate objects?
  - *Improvement:* "We implemented a **Gesture Recognition** module that translates screen-space touch inputs into world-space transformations, restricted by collision boxes to prevent 'ghosting' through walls."

1.3.3 Section: Testing and Validation

This is the most critical addition for a final report. Use the SMART objectives from your proposal to create a "Pass/Fail" or "Result" table.

Key Metrics to Include:

- **Performance Benchmarking:** "The application maintained a consistent **55–60 FPS** on a Samsung S22, though performance dropped to 40 FPS in low-light environments due to increased CPU load from feature-point searching."
- **Drift Analysis:** "In a 10-minute stress test, the virtual object's anchor point drifted by less than 2cm, meeting our accuracy objective."

Chapter 2

Examples: LaTeX Capabilities

2.1 Tables and Structured Data

LaTeX excels at creating professional tables with precise alignment and formatting. Table 2.1 [Tea24] shows typical performance metrics captured during testing phases.

Component	Load (ms)	FPS	Memory (MB)
Rendering Engine	2.5	60	128
Physics Simulation	3.1	55	64
Asset Management	1.8	60	96
UI Framework	0.9	60	32

Table 2.1: Performance Metrics

2.2 Figures and Visual Content

Figures can be embedded using the graphicx package. Figure 2.1 represents a typical workflow diagram (placeholder).

Multiple figures can be arranged side-by-side or stacked vertically using minipage environments, enabling complex document layouts.

2.3 Citations and References

Academic reports benefit from proper citation management. This section demonstrates how to cite sources using LaTeX's bibliography system. According to the documentation [Lam94], LaTeX provides powerful tools for document preparation.

The combination of structured markup and automated formatting ensures consistency across all citations and references throughout the document [Knu84].

2.3.1 Using Biblatex for Advanced Citation Management

The `biblatex` package provides modern, flexible citation management with powerful formatting options. Biblatex offers multiple citation styles and commands:

- `\cite{}`: Parenthetical citations: "This is well documented [Lam94; Knu84]."

- `\textcite{}`: Textual citations: "According to Team [Tea24], LaTeX2e provides robust tools for professional documents."
- `\parencite{}`: Alternative parenthetical style.
- `\footcite{}`: Footnote citations.

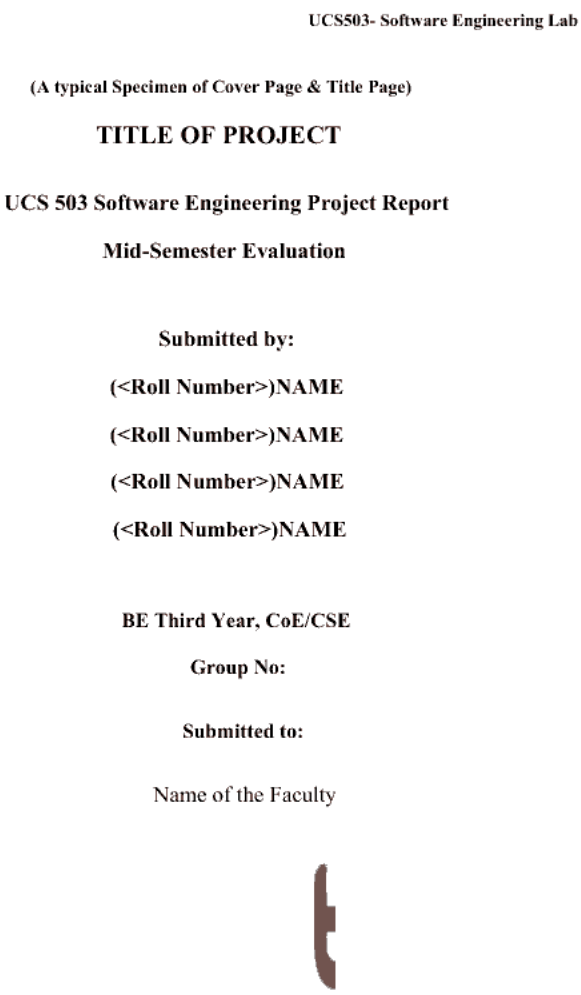
Biblatex requires the `biber` backend, providing superior bibliography management compared to older `natbib` package.

2.3.2 Integrating Zotero with Biblatex

Zotero is a free, open-source reference manager that integrates seamlessly with biblatex via BetterBibTeX. The workflow is straightforward:

1. Install Zotero and the BetterBibTeX extension from `retorque.re`.
2. Create or import your reference library into Zotero.
3. Export your library as a `.bib` file (BibTeX format).
4. Reference the file in your LaTeX preamble using `\addbibresource{references.bib}`.
5. Cite sources using `\cite{}`, `\textcite{}`, or `\footcite{}`.
6. Compile with: `pdflatex` → `biber` → `pdflatex`.

Zotero automatically generates citation keys. BetterBibTeX auto-exports your library to `.bib` file, keeping references synchronized with your Zotero collection automatically.



UCS503- Software Engineering Lab

(A typical Specimen of Cover Page & Title Page)

TITLE OF PROJECT

UCS 503 Software Engineering Project Report

Mid-Semester Evaluation

Submitted by:

(<Roll Number>)NAME

(<Roll Number>)NAME

(<Roll Number>)NAME

(<Roll Number>)NAME

BE Third Year, CoE/CSE

Group No:

Submitted to:

Name of the Faculty

Computer Science and Engineering Department

TIET, Patiala

Figure 2.1: Sample title page layout

Chapter 3

Conclusion

To make this specific to your actual project, start by looking into:

1. **What is the specific title or goal of your project?** (e.g., An AR furniture app? A data analysis tool?)
2. **What was the hardest technical hurdle you faced?** (We'll together write the "Challenges & Mitigations" section for this).

Bibliography

[Knu84] Donald E. Knuth. *The TeXbook*. Computers and Typesetting. Addison-Wesley, 1984 (cit. on p. 11).

[Lam94] Leslie Lamport. *LaTeX: A Document Preparation System*. 2nd. Addison Wesley, 1994 (cit. on p. 11).

[Tea24] LaTeX Project Team. *LaTeX2e: An updated version of LaTeX*. 2024 (cit. on pp. 11, 12).